

Since we want the contents of the XML, you need to apply the 'car' operation to obtain the XML as shown below:

```
(setq xml (car doc))
(book ((id . "compiler")) "
  " (bookinfo nil "
    " (bookbiblio nil "
      " (title nil "My own book!") "
      " (edition nil "First") "
      " (authorgroup nil "
        " ... "
      " ) "
    " ) "
  " ) "
  " (chapter nil "
    " (title nil "Chapter 1") "
    " (para nil "A B C") "
  " ) "
")
```

The 'xml-parse-file' accepts an input file as its argument, and returns the top node of the well-formed XML with all its children. An example with the 'sample.xml' input file is as follows:

```
(xml-parse-file file &optional parse-dtd parse-ns) ;; Syntax

(setq document (car (xml-parse-file "/tmp/sample.xml")))
(book ((id . "compiler")) "
  " (bookinfo nil "
    " (bookbiblio nil "
      " (title nil "My own book!") "
      " (edition nil "First") "
      " (authorgroup nil "
        " ... "
      " ) "
    " ) "
  " ) "
  " (chapter nil "
    " (title nil "Chapter 1") "
    " (para nil "A B C") "
  " ) "
")
```

The *xml-parse-file* function, like the *xml-parse-region* function, actually uses the *xml-parse-buffer* function internally. The source code of the *xml-parse-file* function is given below:

```
(defun xml-parse-file (file &optional parse-dtd parse-ns)
  (with-temp-buffer
    (insert-file-contents file)
    (xml--parse-buffer parse-dtd parse-ns)))
```

The *xml-node-name* function accepts a node as an argument and retrieves its name as indicated below:

```
(xml-node-name node) ;; Syntax

(xml-node-name xml)
book
```

The source code of the *xml-node-name* function is as follows:

```
(defsubst xml-node-name (node)
  (car node))
```

The attributes of a node can be returned using the *xml-node-attributes* API. For example:

```
(xml-node-attributes node) ;; Syntax

(xml-node-attributes xml)
((id . "compiler"))
```

The implementation of the *xml-node-attributes* function is as follows:

```
(defsubst xml-node-attributes (node)
  "Return the list of attributes of NODE.
The list can be nil."
  (nth 1 node))
```

The *xml-node-children* function takes a node as its argument, and obtains all the children of the node as follows:

```
(xml-node-children node) ;; Syntax

(xml-node-children xml)
(
  " (bookinfo nil "
    " (bookbiblio nil "
      " (title nil "My own book!") "
      " (edition nil "First") "
      " (authorgroup nil "
        " ... "
      " ) "
    " ) "
  " ) "
  " (chapter nil "
    " (title nil "Chapter 1") "
    " (para nil "A B C") "
  " ) "
)
```